

LINEAR REGRESSION AND TREE REGRESSION

Análise de Dados em Informática
Licenciatura em Engenharia Informática
ISEP/IPP

Ana Maria Madureira
2025/2026

Fontes:

- Tom Mitchell, Machine Learning, McGraw-Hill, 1997.
- Catarina Silva e Bernardete Ribeiro, Aprendizagem Computacional em Engenharia, Imprensa da Universidade de Coimbra, 2018

38

38

SIMPLE LINEAR REGRESSION

- **Regression analysis** is one of the most widely used methods for prediction.
 - it is applied whenever we have a causal relationship between 2 or more variables.
 - it is a valuable tool for understanding and quantifying the relationship between two variables.

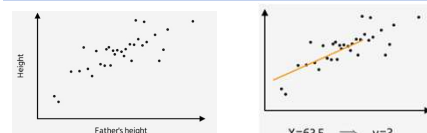
The Simple **Linear Regression** model:

$$Y = \theta_0 + \theta_1 * x + \epsilon.$$

Y is the variable we are trying to predict and is called the *dependent variable*. X is an *independent variable*.

A linear regression is a statistical model that analyzes the relationship between a response variable (often called Y) and one or more variables and their interactions (often called X or explanatory variables, or predictors).

1. Importing the Python libraries we will use
2. Getting the data
3. Defining x values (the input variable) and y values (the output variable)
4. Machine Learning: fitting the model
5. Interpreting the results (coefficient, intercept) and calculating the accuracy of the model
6. Visualization (plotting a graph)



want to predict the person's height if his father is 63.5 inches tall. Just pick a point from the line that corresponds to $X=63.5$, and its y value is our prediction. The model predicts the person to be 64.3 inches tall.

39

39

SIMPLE LINEAR REGRESSION

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from pandas.core.common import random_state
from sklearn.linear_model import LinearRegression

# Step 2: Load the dataset
df_sal = pd.read_csv('salary_data.csv')
df_sal.head()

# Step 3: Data analysis
# Describe data
df_sal.describe()
# Data distribution
plt.title('Salary distribution Plot')
sns.histplot(df_sal['salary'], kde=True)
plt.show()

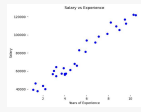
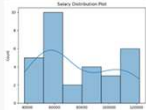
# Relationship between Salary and Experience
plt.scatter(df_sal['years_experience'], df_sal['salary'], color='blue')
plt.title('Salary vs Experience')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.box(False)
plt.show()

# Correlation Matrix
df_sal.corr()
sns.heatmap(df_sal.corr(), annot=True)

# Step 4: Split the dataset into dependent/independent variables
# Experience (X) is the independent variable
# Salary (Y) is dependent on experience
X = df_sal.iloc[:, 1] # independent
y = df_sal.iloc[:, 2] # dependent

# Step 4: Split data into train/test sets
# Splitting dataset into test/train set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 5: Train the regression model
# Pass the X_train and y_train data into the regressor model by regressor.fit to train the model with our training data.
# Regressor model
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```



of Ex

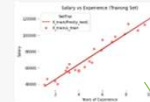
```
# Step 6: Predict the result
y_pred_test = regressor.predict(X_test) # predicted value of y_test
y_pred_train = regressor.predict(X_train) # predicted value of y_train

# Step 7: Plot the training and test results
# Prediction on training set
plt.scatter(X_train, y_train, color='lightcoral')
plt.plot(X_train, y_pred_train, color='firebrick')
plt.title('Salary vs Experience (Training Set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend(['X_train/Pred(y_test)', 'X_train/y_train'], title='Sal/Exp', loc='best', facecolor='white')
plt.box(False)
plt.show()

# Plot test set data vs predictions
# Prediction on test set
plt.scatter(X_test, y_test, color='lightcoral')
plt.plot(X_train, y_pred_train, color='firebrick')
plt.title('Salary vs Experience (Test Set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend(['X_train/Pred(y_test)', 'X_train/y_train'], title='Sal/Exp', loc='best', facecolor='white')
plt.box(False)
plt.show()

# Regressor coefficients and intercept
print("y=", regressor.intercept_, "+", regressor.coef_, "x")

# Evaluate the model
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
y_pred = regressor.predict(X_test)
MAE = np.mean(abs(y_test - y_pred))
print("MAE on training set: {:.3f}".format(MAE))
```



As we can see, most of the observations are close to the regression line, hence we can say our Linear Regression is a good model and able to make good predictions.

$y = [26780.09915063] + [[9312.57512673]] x$
MAE on training set: 2446.172

40

TREE REGRESSION

```
# Tree regression
from sklearn.tree import DecisionTreeRegressor
from sklearn import metrics
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
tree = DecisionTreeRegressor(random_state=42, max_depth=6, min_samples_split=3)

model = tree.fit(X_train, y_train)

y_pred = tree.predict(X_train)
y_pred1 = tree.predict(X_test)

MAE1 = metrics.mean_absolute_error(y_train, y_pred)
MAE2 = metrics.mean_absolute_error(y_test, y_pred1)

print("MAE on training set: {:.3f}".format(MAE1))
print("MAE on test set: {:.3f}".format(MAE2))

# Regression tree visualization
from sklearn import tree

tree.plot_tree(model,
               feature_names = list(df_sal.columns),
               class_names = list(df_sal['salary']),
               filled=True,
               fontsize=6);

# the model:
from sklearn.tree import export_text
print(export_text(model, show_weights=True))
```

41

40

41

MULTIPLE LINEAR REGRESSION

When a regression takes into account two or more predictors to create the linear regression, it's called multiple linear regression (MLR)

- For MLR, the dependent or target variable(Y) must be the continuous/real, but the predictor or independent variable may be of continuous or categorical form.
- Each feature variable must model the linear relationship with the dependent variable.
- MLR tries to fit a regression line through a multidimensional space of data-points.

Assumptions for Multiple Linear Regression:

- A **linear relationship** should exist between the Target and predictor variables.
- The regression residuals must be **normally distributed**.
- MLR assumes little or **no multicollinearity** (correlation between the independent variable) in data.

```
from sklearn import linear_model
df = pandas.read_csv("data.csv")
X = df[['Weight', 'Volume']]
y = df['CO2']

regr = linear_model.LinearRegression()
regr.fit(X, y)

#predict the CO2 emission of a car where the weight is 2300kg, and the volume is 1300cm³:
predictedCO2 = regr.predict([[2300, 1300]])
print(predictedCO2)
```

This model predicted that a car with 1.3 liter engine, and a weight of 2300 kg, will release approximately [107.2087328] grams of CO2 for each kilometer it drives.

```
from sklearn import linear_model
df = pandas.read_csv("data.csv")
X = df[['Weight', 'Volume']]
y = df['CO2']

regr = linear_model.LinearRegression()
regr.fit(X, y)
print(regr.coef_)
```

The coefficient values of weight and volume.
Weight: 0.00755095
Volume: 0.00780526
- if the weight increase by 1kg, the CO2 emission increases by 0.00755095g.
- if the engine size (Volume) increases by 1 cm³, the CO2 emission increases by 0.00780526g.

The model predicted that a car with 1.3 liter engine, and a weight of 3300 kg, will release approximately 115 grams of CO2 for every kilometer it drives.

Which shows that the coefficient of 0.00755095 is correct:

$$107.2087328 + (1000 * 0.00755095) = 114.75968$$

42

REGRESSION MODEL EVALUATION METRICS

Regression model evaluation metrics

The **MSE**, **MAE**, **RMSE**, and **R-Squared** metrics are mainly used to evaluate the prediction error rates and model performance in regression analysis.

- MAE (Mean absolute error)** represents the difference between the original and predicted values extracted by averaged the absolute difference over the data set.
- MSE (Mean Squared Error)** represents the difference between the original and predicted values extracted by squared the average difference over the data set.
- RMSE (Root Mean Squared Error)** is the error rate by the square root of MSE.
- R-squared (Coefficient of determination)** represents the coefficient of how well the values fit compared to the original values. The value from 0 to 1 interpreted as percentages. The higher the value is, the better the model is.

43

43